Syed Naziya Parveen

Batch-42

2303A52394

Task 1: Employee Dataset Cleaning

Prompt: A company maintains an employee dataset containing employee ID, name, designation, salary, experience, and employment type. The dataset may contain missing values, duplicate records, and inconsistent text formats.Clean and preprocess the dataset by handling missing values in salary, designation, and experience, removing duplicate employee records, standardizing text values to lowercase, and encoding categorical variables such as designation and employment_type.

The final output should be a clean dataset suitable for HR analytics and further processing.

Dataset Creation:

```
#TASK -1
import pandas as pd

data = {
    "employee_id": [1, 2, 3, 4, 5, 5],
    "name": ["Alice", "Bob", "Charlie", "David", "Eve", "Eve"],
    "salary": [50000, None, 60000, 55000, 50000, 50000],
    "designation": ["Manager", "Developer", None, "Developer", "Manager", "Manager"],
    "experience": [5, 3, 4, None, 5, 5],
    "employment_type": ["Full-time", "Part-time", "Full-time", "Contract", "Full-time", "Full-time"]
}

df = pd.DataFrame(data)

df.to_csv("employee_data.csv", index=False)

print("CSV file created!")
```

```
from google.colab import files
files.download("employee_data.csv")
```

Code:

```python
import pandas as pd

# ------------------------------
# STEP 1: LOAD CSV FILE
# ------------------------------
df = pd.read_csv("employee_data.csv")

print("Original Data:\n")
print(df)

# ------------------------------
# STEP 2: HANDLE MISSING VALUES
# ------------------------------

# Fill salary with median
df['salary'].fillna(df['salary'].median(), inplace=True)

# Fill experience with median
df['experience'].fillna(df['experience'].median(), inplace=True)

# Fill designation with most frequent value (mode)
df['designation'].fillna(df['designation'].mode()[0], inplace=True)

# ------------------------------
# STEP 3: REMOVE DUPLICATES
# ------------------------------

df = df.drop_duplicates(subset='employee_id')

# ------------------------------
# STEP 4: STANDARDIZE TEXT
# ------------------------------

df['designation'] = df['designation'].str.lower().str.strip()
```

```python
# ------------------------------
# STEP 5: ENCODE CATEGORICAL DATA
# ------------------------------

df = pd.get_dummies(df, columns=['designation', 'employment_type'])

# ------------------------------
# STEP 6: SAVE CLEAN DATASET
# ------------------------------

df.to_csv("clean_employee_data.csv", index=False)

print("\nCleaned Data:\n")
print(df)
```

Input:

```
Original Data:

   employee_id     name   salary designation  experience employment_type
0            1    Alice  50000.0     Manager         5.0       Full-time
1            2      Bob      NaN   Developer         3.0       Part-time
2            3  Charlie  60000.0         NaN         4.0       Full-time
3            4    David  55000.0   Developer         NaN        Contract
4            5      Eve  50000.0     Manager         5.0       Full-time
5            5      Eve  50000.0     Manager         5.0       Full-time
```

Output:

```
Cleaned Data:

   employee_id     name   salary  experience  designation_developer  \
0            1    Alice  50000.0         5.0                  False
1            2      Bob  50000.0         3.0                   True
2            3  Charlie  60000.0         4.0                  False
3            4    David  55000.0         5.0                   True
4            5      Eve  50000.0         5.0                  False

   designation_manager  employment_type_Contract  employment_type_Full-time  \
0                 True                     False                       True
1                False                     False                      False
2                 True                     False                       True
3                False                      True                      False
4                 True                     False                       True

   employment_type_Part-time
0                      False
1                       True
2                      False
3                      False
4                      False
```

Justification:

Employee datasets often contain missing values, duplicates, and inconsistent formats. Cleaning ensures data accuracy and reliability. Handling missing values prevents loss of important information. Removing duplicates avoids redundancy and incorrect analysis. Standardizing and encoding data makes it suitable for HR analytics and machine learning models

Task 2 Prompt – Student Performance Data Preprocessing

A dataset contains student academic performance data including exam_date and total_marks. The dataset requires preprocessing to extract useful features and improve data quality.Convert the exam_date column into datetime format, extract additional features such as month, day, or semester, normalize the total_marks column, and detect and handle outliers using statistical methods such as IQR or Z-score.

The final output should be a transformed dataset with time-based features and normalized marks.

Dataset Creation:

```python
import pandas as pd

data = {
    "student_id": list(range(1, 13)),
    "name": ["Alice","Bob","Charlie","David","Eve","Frank",
             "Grace","Helen","Ian","Jack","Kate","Leo"],
    "exam_date": [
        "2025-01-15","2025-02-20","2025-03-10","2025-04-05",
        "2025-05-18","2025-06-25","2025-07-12","2025-08-30",
        "2025-09-14","2025-10-01","2025-11-22","2025-12-10"
    ],
    "total_marks": [85,78,92,60,88,45,95,120,70,65,30,85]
}

df = pd.DataFrame(data)

df.to_csv("student_data.csv", index=False)

print("Dataset created!")
```

```
Dataset created!
```

```python
from google.colab import files
files.download("student_data.csv")
```

Code:

```python
import pandas as pd
import numpy as np

# ------------------------------
# STEP 1: LOAD DATASET
# ------------------------------
df = pd.read_csv("student_data.csv")

print("Original Data:\n")
print(df)

# ------------------------------
# STEP 2: CONVERT exam_date → datetime
# ------------------------------
df['exam_date'] = pd.to_datetime(df['exam_date'])

# ------------------------------
# STEP 3: EXTRACT FEATURES
# ------------------------------
df['month'] = df['exam_date'].dt.month
df['day'] = df['exam_date'].dt.day

# Semester logic (simple)
df['semester'] = df['month'].apply(lambda x: 1 if x <= 6 else 2)

# ------------------------------
# STEP 4: NORMALIZE total_marks
# ------------------------------
df['total_marks_normalized'] = (
    (df['total_marks'] - df['total_marks'].min()) /
    (df['total_marks'].max() - df['total_marks'].min())
)

# ------------------------------
# STEP 5: OUTLIER DETECTION (IQR METHOD)
# ------------------------------
```

```python
# ------------------------------
# STEP 5: OUTLIER DETECTION (IQR METHOD)
# ------------------------------
Q1 = df['total_marks'].quantile(0.25)
Q3 = df['total_marks'].quantile(0.75)
IQR = Q3 - Q1

lower_bound = Q1 - 1.5 * IQR
upper_bound = Q3 + 1.5 * IQR

# Remove outliers
df = df[(df['total_marks'] >= lower_bound) & (df['total_marks'] <= upper_bound)

# ------------------------------
# STEP 6: SAVE CLEAN DATA
# ------------------------------
df.to_csv("clean_student_data.csv", index=False)

print("\nProcessed Data:\n")
print(df)
```

Input:

```
•••  Original Data:

     student_id     name   exam_date  total_marks
0             1    Alice  2025-01-15           85
1             2      Bob  2025-02-20           78
2             3  Charlie  2025-03-10           92
3             4    David  2025-04-05           60
4             5      Eve  2025-05-18           88
5             6    Frank  2025-06-25           45
6             7    Grace  2025-07-12           95
7             8    Helen  2025-08-30          120
8             9      Ian  2025-09-14           70
9            10     Jack  2025-10-01           65
10           11     Kate  2025-11-22           30
11           12      Leo  2025-12-10           85
```

Output:

```
Processed Data:

    student_id     name  exam_date  total_marks  month  day  semester  \
0            1    Alice 2025-01-15           85      1   15         1
1            2      Bob 2025-02-20           78      2   20         1
2            3  Charlie 2025-03-10           92      3   10         1
3            4    David 2025-04-05           60      4    5         1
4            5      Eve 2025-05-18           88      5   18         1
5            6    Frank 2025-06-25           45      6   25         1
6            7    Grace 2025-07-12           95      7   12         2
7            8    Helen 2025-08-30          120      8   30         2
8            9      Ian 2025-09-14           70      9   14         2
9           10     Jack 2025-10-01           65     10    1         2
10          11     Kate 2025-11-22           30     11   22         2
11          12      Leo 2025-12-10           85     12   10         2

    total_marks_normalized
0                 0.611111
1                 0.533333
2                 0.688889
3                 0.333333
4                 0.644444
5                 0.166667
6                 0.722222
7                 1.000000
8                 0.444444
9                 0.388889
10                0.000000
11                0.611111
```

Justification:

Student performance data includes time-based and numerical attributes that need transformation. Converting dates into datetime format enables extraction of useful temporal features. Normalizing marks ensures uniform scaling for better analysis. Handling outliers prevents skewed results. These steps improve data quality for accurate predictions and insights.

Task 3 Prompt – Student Health Data Preparation

A dataset contains student health records including height, weight, BMI, medical_condition, and fitness_status. The dataset may include missing values and categorical attributes. Clean and preprocess the dataset by handling missing values in height, weight, and BMI, encoding categorical variables such as medical_condition and fitness_status, scaling numerical features using standard scaling techniques, and splitting the dataset into training and testing sets.

The final output should be a preprocessed dataset ready for predictive health analysis.

Dataset Creation:

```python
import pandas as pd

# Create dataset
data = {
    "student_id": [1,2,3,4,5,6,7,8,9,10],
    "height": [160,165,170,155,180,None,172,168,175,162],
    "weight": [55,60,None,50,75,65,68,70,80,58],
    "BMI": [21.5,22.0,24.2,None,23.1,25.0,22.9,None,26.1,22.1],
    "medical_condition": ["None","Diabetes","None","Asthma","None","Obesity","None","Diabetes","Obesity","None"],
    "fitness_status": ["Fit","Average","Unfit","Fit","Average","Unfit","Fit","Average","Unfit","Fit"]
}

# Convert to DataFrame
df = pd.DataFrame(data)

# Save as CSV
df.to_csv("student_health.csv", index=False)

print("CSV file created successfully!")
```

```
CSV file created successfully!
```

```python
from google.colab import files
files.download("student_health.csv")
```

Code:

```python
import pandas as pd
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import train_test_split

# ------------------------------
# STEP 1: LOAD DATASET
# ------------------------------
df = pd.read_csv("student_health.csv")

print("Original Data:\n")
print(df)

# ------------------------------
# STEP 2: HANDLE MISSING VALUES
# ------------------------------
df['height'].fillna(df['height'].mean(), inplace=True)
df['weight'].fillna(df['weight'].mean(), inplace=True)
df['BMI'].fillna(df['BMI'].mean(), inplace=True)

df['medical_condition'].fillna(df['medical_condition'].mode()[0], inplace=True)
df['fitness_status'].fillna(df['fitness_status'].mode()[0], inplace=True)

# ------------------------------
# STEP 3: ENCODE CATEGORICAL FEATURES
# ------------------------------
df = pd.get_dummies(df, columns=['medical_condition', 'fitness_status'])

# ------------------------------
# STEP 4: SCALE NUMERICAL FEATURES
# ------------------------------
scaler = StandardScaler()
df[['height', 'weight', 'BMI']] = scaler.fit_transform(df[['height', 'weight', 'BMI']])
```

```python
# ------------------------------
# STEP 5: SPLIT DATASET
# ------------------------------
X = df.drop('student_id', axis=1)
y = df['BMI']

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# ------------------------------
# STEP 6: OUTPUT
# ------------------------------
print("\nProcessed Data:\n")
print(df)

print("\nTraining Shape:", X_train.shape)
print("Testing Shape:", X_test.shape)
```

Input:

```
•••  Original Data:

       student_id  height  weight   BMI medical_condition fitness_status
    0           1   160.0    55.0  21.5               NaN            Fit
    1           2   165.0    60.0  22.0          Diabetes        Average
    2           3   170.0     NaN  24.2               NaN          Unfit
    3           4   155.0    50.0   NaN            Asthma            Fit
    4           5   180.0    75.0  23.1               NaN        Average
    5           6     NaN    65.0  25.0           Obesity          Unfit
    6           7   172.0    68.0  22.9               NaN            Fit
    7           8   168.0    70.0   NaN          Diabetes        Average
    8           9   175.0    80.0  26.1           Obesity          Unfit
    9          10   162.0    58.0  22.1               NaN            Fit
```

Output:

```
...
Processed Data:

   student_id        height    weight           BMI  medical_condition_Asthma  \
0           1 -1.065427e+00 -1.098832 -1.385199e+00                     False
1           2 -3.498416e-01 -0.523862 -1.013333e+00                     False
2           3  3.657434e-01  0.000000  6.228746e-01                     False
3           4 -1.781012e+00 -1.673802 -2.642263e-15                      True
4           5  1.796913e+00  1.201049 -1.952293e-01                     False
5           6  4.067630e-15  0.051108  1.217859e+00                     False
6           7  6.519775e-01  0.396091 -3.439755e-01                     False
7           8  7.950945e-02  0.626079 -2.642263e-15                     False
8           9  1.081328e+00  1.776019  2.035963e+00                     False
9          10 -7.791926e-01 -0.753850 -9.389602e-01                     False

   medical_condition_Diabetes  medical_condition_Obesity  \
0                        True                      False
1                        True                      False
2                        True                      False
3                       False                      False
4                        True                      False
5                       False                       True
6                        True                      False
7                        True                      False
8                       False                       True
9                        True                      False

   fitness_status_Average  fitness_status_Fit  fitness_status_Unfit
0                   False                True                 False
1                    True               False                 False
2                   False               False                  True
3                   False                True                 False
4                    True               False                 False
5                   False               False                  True
6                   False                True                 False
7                    True               False                 False
8                   False               False                  True
9                   False                True                 False

Training Shape: (8, 9)
Testing Shape: (2, 9)
```

Justification:

Student health datasets may contain missing values and categorical attributes. Handling missing data ensures completeness and avoids bias. Encoding categorical variables converts them into numerical form for model usage. Scaling numerical features ensures fair contribution of all variables. Splitting data helps evaluate model performance effectively.

Task 4 Prompt – Ride-Sharing Data Cleaning

A ride-sharing company maintains trip data including pickup location, drop location, fare amount, and driver ratings. The dataset may contain inconsistencies, missing values, and duplicate records.

Clean and preprocess the dataset by standardizing pickup and drop location names, filling missing fare values using median, removing duplicate ride entries, normalizing driver ratings, and generating a summary of improvements after cleaning.

The final output should be a clean dataset suitable for operational analysis and decision-making.

Dataset Creation:

```python
import pandas as pd

# Create dataset
data = {
    "ride_id": [1,2,3,4,5,6,7,8],
    "pickup_location": ["Chennai ","chennai","Velachery","Velachery","OM
    "drop_location": ["T Nagar","T nagar","Adyar","Adyar","Perungudi","p
    "fare": [250,None,180,180,None,300,220,220],
    "driver_rating": [4.5,4.2,4.8,4.8,3.9,4.0,4.6,4.6]
}

# Convert to DataFrame
df = pd.DataFrame(data)

# Save as CSV
df.to_csv("rides_data.csv", index=False)

print("CSV file created successfully!")
```

```
CSV file created successfully!
```

```python
from google.colab import files
files.download("rides_data.csv")
```

Code:

```python
import pandas as pd

# -------------------------------
# STEP 1: LOAD DATASET
# -------------------------------
df = pd.read_csv("rides_data.csv")

print("Original Data:\n")
print(df)

# -------------------------------
# STEP 2: STANDARDIZE LOCATIONS
# -------------------------------
df['pickup_location'] = df['pickup_location'].str.lower().str.strip()
df['drop_location'] = df['drop_location'].str.lower().str.strip()

# -------------------------------
# STEP 3: HANDLE MISSING FARE
# -------------------------------
df['fare'].fillna(df['fare'].median(), inplace=True)

# -------------------------------
# STEP 4: REMOVE DUPLICATES
# -------------------------------
df = df.drop_duplicates()

# -------------------------------
# STEP 5: NORMALIZE DRIVER RATINGS
# -------------------------------
df['driver_rating'] = (
    (df['driver_rating'] - df['driver_rating'].min()) /
    (df['driver_rating'].max() - df['driver_rating'].min())
)
```

```python
)

# -------------------------------
# STEP 6: SUMMARY OF IMPROVEMENTS
# -------------------------------
print("\nSummary:")
print("Total rows after cleaning:", len(df))
print("Missing values remaining:\n", df.isnull().sum())

# -------------------------------
# STEP 7: SAVE CLEAN DATA
# -------------------------------
df.to_csv("clean_rides_data.csv", index=False)

print("\nCleaned Data:\n")
print(df)
```

Input:

```
Original Data:

   ride_id pickup_location drop_location   fare  driver_rating
0        1        Chennai        T Nagar  250.0            4.5
1        2         chennai       T nagar    NaN            4.2
2        3       Velachery         Adyar  180.0            4.8
3        4       Velachery         Adyar  180.0            4.8
4        5             OMR     Perungudi    NaN            3.9
5        6            omr      perungudi  300.0            4.0
6        7      Anna Nagar        Egmore  220.0            4.6
7        8     Anna nagar         Egmore  220.0            4.6

Summary:
Total rows after cleaning: 8
Missing values remaining:
 ride_id            0
pickup_location    0
drop_location      0
fare               0
driver_rating      0
dtype: int64
```

Output:

```
Cleaned Data:

   ride_id pickup_location drop_location   fare  driver_rating
0        1         chennai       t nagar  250.0       0.666667
1        2         chennai       t nagar  220.0       0.333333
2        3       velachery         adyar  180.0       1.000000
3        4       velachery         adyar  180.0       1.000000
4        5             omr     perungudi  220.0       0.000000
5        6             omr     perungudi  300.0       0.111111
6        7      anna nagar        egmore  220.0       0.777778
7        8      anna nagar        egmore  220.0       0.777778
```

Justification:

Ride-sharing datasets often contain inconsistent text, missing values, and duplicates. Standardizing location names ensures uniformity in analysis. Filling missing fare values using median maintains data integrity. Removing duplicates avoids redundancy in records. Normalizing ratings ensures consistent comparison and improves data quality for analysis.